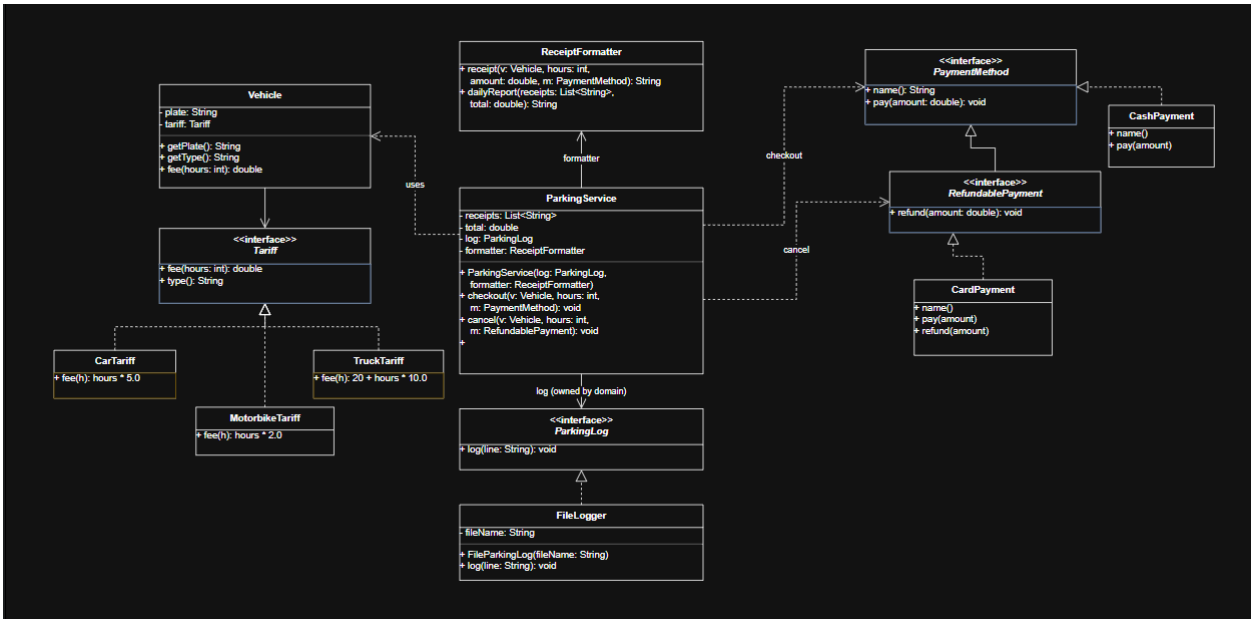
2/

Principles:	Class
SRP	Parking garage: it has too many actors that could change it
Open/Close	paring garage: the switch causes changes on already done classes
LSP	CashPayment : method non implemented and unecessary for the client (refund())
ISP	PaymentMethod : Fat interface
DIP	Parking garage: depends on file logger(it gets created each time with every new sentence)

3/ a. change in the class parkingGarage in the class fee by adding a new switch
b. Creating a new class implementing paymentMethod
c. changing the file logger
d. ParkingGarage would be concerned for the report in csv format using the filelogger to fetch the data

4/ if we remove the test for instance of cashPayment it will try to execute refund on instances of cashpayment who throws UnsupportedOperationException, it will violate the openclose
principle it is a polymorphism issue

6/Class diagram representing the refactored project:



8. With your design, what happens at compile time if a developer tries to cancel a cash payment through ParkingService? Compare with question 4

- the developper will not be able to do it since cancel supports only the methods who implements RefundablePayment and since our cashPayment only implements PaymentMethod , therefore a compile-time error will occur